

Sistema de Control en Tiempo Real de un Ventilador mediante una Interfaz Web usando ESP32

Brayan Manuel Gallego Ocampo
Luis Fernando Gamba Bedoya
Facultad de Ingeniería Electrónica
Universidad Nacional de Colombia
Manizales - Caldas, Colombia
lgambab@unal.edu.co, bgallego@unal.edu.co

Resumen—Este proyecto presenta el diseño e implementación de un sistema embebido de control en tiempo real para un ventilador eléctrico mediante un microcontrolador ESP32 actuando como servidor web. El sistema permite controlar la velocidad, encendido y modos de operación a través de una interfaz web desarrollada en HTML, CSS y JavaScript, comunicándose mediante HTTP y JSON. Se integran sensores NTC para medición de temperatura, un sensor PIR para detección de presencia, lógica de programación horaria mediante NVS y control PWM de velocidad. El sistema opera en tres modos: manual, automático por temperatura y automático por horario, garantizando un funcionamiento flexible, robusto y con respuesta inmediata. El proyecto combina conceptos de tiempo real, IoT, electrónica digital y diseño web, mostrando la viabilidad del ESP32 como plataforma de automatización de bajo costo.

Index Terms—ESP32, sistemas en tiempo real, IoT, HTTP, JSON, PWM, NTC, PIR, SNTP, NVS.

I. INTRODUCCIÓN

Los sistemas embebidos conectados a Internet han impulsado aplicaciones que integran sensado, control y comunicación en tiempo real. En este contexto, el ESP32 se ha consolidado como una plataforma versátil gracias a su conectividad Wi-Fi, capacidades multitarea mediante FreeRTOS y periféricos analógicos y digitales.

El presente trabajo desarrolla un sistema completo para el control de un ventilador a través de una interfaz web, permitiendo modificar su velocidad, encendido y modos de operación desde cualquier dispositivo conectado a la red local. La comunicación se basa en HTTP/JSON y el sistema integra sensores para la toma de decisiones autónomas: un NTC para temperatura y un sensor PIR para la detección de presencia.

La flexibilidad del diseño permite combinar automatización, confort y eficiencia energética, evidenciando cómo tecnologías IoT pueden implementarse en tareas cotidianas mediante sistemas en tiempo real de bajo costo.

II. PLANTEAMIENTO DEL PROBLEMA

Los ventiladores convencionales carecen de mecanismos de automatización, monitoreo remoto y adaptación dinámica al entorno. Generalmente operan en uno o dos niveles fijos y requieren intervención manual. Esto genera:

- Falta de eficiencia energética.
- Incapacidad para ajustarse a cambios de temperatura.
- Imposibilidad de operación basada en horarios programados.
- Ausencia de detección de presencia.

Por tanto, se requiere un sistema capaz de:

1. Controlar el ventilador remotamente.
2. Ajustar la operación de acuerdo con temperatura ambiental.
3. Activarse únicamente cuando detecte presencia (PIR).
4. Programar encendido y apagado automático según horarios.
5. Proveer respuesta inmediata y comportamiento en tiempo real.

III. OBJETIVOS

III-A. Objetivo General

Diseñar e implementar un sistema de control en tiempo real para un ventilador a través de una interfaz web usando un ESP32 como servidor embebido.

III-B. Objetivos Específicos

- Implementar un servidor web y API REST en el ESP32.
- Diseñar una interfaz web intuitiva para control del ventilador.
- Establecer comunicación eficiente mediante HTTP y JSON.
- Implementar control de velocidad por PWM.
- Integrar sensores NTC (temperatura) y PIR (movimiento).
- Desarrollar modos manual, automático por temperatura y por horario.
- Almacenar configuraciones en NVS para persistencia.
- Sincronizar el reloj mediante SNTP para programación horaria.

IV. ALCANCE

El sistema permite el control local del ventilador mediante red WiFi privada. No incluye acceso remoto por Internet ni integración con servicios en la nube. El sistema opera sobre

un solo ventilador y un conjunto fijo de sensores. La interfaz web se limita al control general, configuración de modos, visualización de temperatura y gestión de horarios.

V. MARCO TEÓRICO

V-A. ESP32 como sistema en tiempo real

El ESP32 incorpora un sistema operativo en tiempo real (FreeRTOS), soporte multitarea, WiFi y ADCs de 12 bits, facilitando la integración de sensores analógicos, comunicación y control con baja latencia. Sus timers y periféricos PWM (LEDC) permiten control preciso de actuadores como ventiladores.

V-B. PWM

El control por modulación por ancho de pulso (PWM) se emplea para ajustar la velocidad del ventilador. El ESP32 usa el periférico LEDC, que permite frecuencias estables y resolución suficiente para control suave.

V-C. Sensor NTC

El NTC se utiliza en un divisor de voltaje y su comportamiento se modela mediante la ecuación Beta:

$$T = \left(\frac{1}{T_0 + 273,15} + \frac{1}{B} \ln \left(\frac{R}{R_0} \right) \right)^{-1} - 273,15$$

El sistema convierte el valor ADC en mV, calcula la resistencia y determina la temperatura en tiempo real.

V-D. Sensor PIR

El sensor PIR detecta radiación infrarroja asociada a movimiento humano. Su salida digital se maneja como interrupción mediante GPIO con ISR, notificando eventos a una cola FreeRTOS.

V-E. HTTP y JSON

La comunicación web se implementa en formato JSON debido a su simplicidad para representar estructuras clave-valor. El ESP32 actúa como servidor HTTP manejando rutas como:

- POST /api/register
- GET /api/registers
- DELETE /api/register/{id}

V-F. NVS y persistencia

La memoria NVS permite almacenar configuraciones persistentes como horarios programados, en formato JSON.

V-G. SNTP

El ESP32 sincroniza hora y fecha mediante SNTP para asegurar precisión en la ejecución de horarios programados.

VI. ARQUITECTURA DEL SISTEMA

VI-A. Componentes principales

- ESP32 como servidor web en tiempo real.
- Ventilador controlado por transistor y PWM.
- Sensor NTC para temperatura.
- Sensor PIR para detección de presencia.
- Interfaz web HTML + JavaScript.
- Módulo de registros almacenados en NVS.

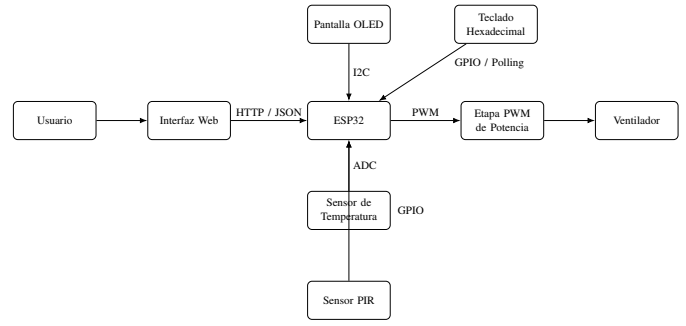


Figura 1. Diagrama de bloques.

VI-B. Diagrama General

Usuario → Página Web → HTTP/JSON → ESP32 → PWM → Ventilador

VI-C. Flujo de Datos

- El usuario cambia un modo o velocidad.
- JavaScript envía un JSON al ESP32.
- El ESP32 procesa la solicitud y actualiza PWM.
- La interfaz consulta periódicamente estado/temperatura.

VI-D. Microarquitectura del ESP32

El ESP32 incorpora una microarquitectura basada en dos núcleos Xtensa LX6 con arquitectura Harvard modificada, permitiendo la ejecución paralela de tareas bajo FreeRTOS. Cada núcleo dispone de unidad de multiplicación-acumulación (MAC), coprocesador, pipeline de tres etapas y manejo avanzado de interrupciones mediante el sistema de prioridades del nivel EXTI. El microcontrolador integra periféricos internos como temporizadores generales, ADC de 12 bits, comparadores de PWM, módulos UART/SPI/I2C y motor WiFi/Bluetooth dedicado. Esta arquitectura permite un procesamiento eficiente y determinista para aplicaciones de control en tiempo real.

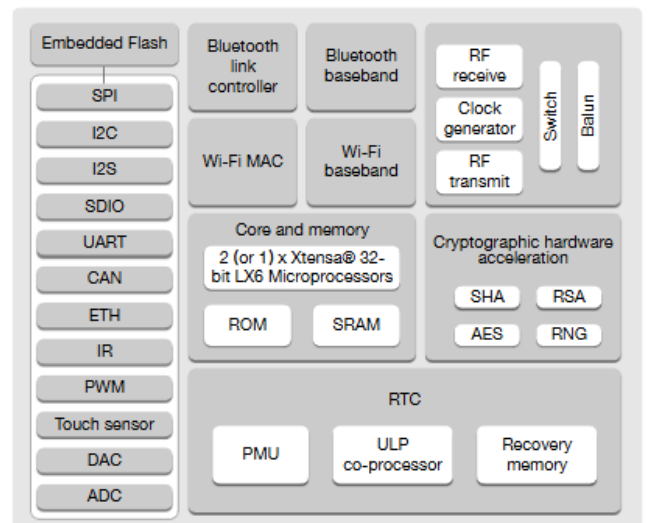


Figura 2. Microarquitectura interna del ESP32.

VII. DESCRIPCIÓN DEL HARDWARE

El hardware del sistema está compuesto por un microcontrolador ESP32 responsable de la adquisición de datos y el control del ventilador. El sistema integra un sensor NTC para la medición de temperatura y un sensor PIR para la detección de presencia, los cuales se conectan directamente a los pines de entrada del ESP32. El ventilador es accionado mediante una etapa de potencia controlada por señal PWM generada por el microcontrolador. La alimentación se realiza mediante una fuente de 5 V y el montaje se efectúa en protoboard para facilitar el desarrollo y pruebas. En conjunto, estos elementos permiten la supervisión del entorno y el accionamiento dinámico del ventilador.

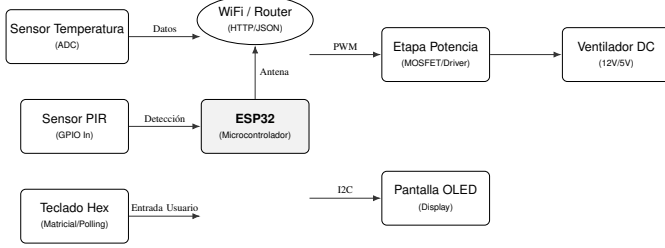


Figura 3. Diagrama de bloques funcional del sistema.

- Microcontrolador ESP32-WROOM.
- Ventilador DC controlado mediante transistor MOSFET.
- NTC 10k para medición de temperatura.
- Sensor PIR HC-SR501.
- Protoboard, cables y fuente 5V.

VIII. FUNCIONALIDADES ADICIONALES

VIII-A. Módulo de visualización local (OLED)

Se integrará una pantalla OLED al sistema con el objetivo de mostrar información relevante como la temperatura actual, la hora obtenida por NTP y la velocidad del ventilador. Este módulo permitirá una supervisión local sin depender de la interfaz web, ofreciendo una vista rápida del estado del sistema, además de un led rgb para confirmar la conexión con la pantalla y hacer saber al usuario de su correcto funcionamiento.

VIII-B. Teclado hexadecimal para autenticación

Se añadirá un teclado hexadecimal que actuará como mecanismo de autenticación para acceder a las funciones de la pantalla OLED. El usuario deberá ingresar una clave válida para habilitar la visualización de los datos, proporcionando una capa adicional de seguridad en el acceso local.

VIII-C. Integración con el sistema principal

Ambos módulos se integrarán con el ESP32 mediante GPIO y tareas específicas en FreeRTOS. La pantalla OLED recibirá actualizaciones periódicas desde el sistema central, mientras que el teclado se gestionará mediante polling (lectura escaneada).

IX. DESCRIPCIÓN DEL SOFTWARE

El software está basado en el framework ESP-IDF y utiliza FreeRTOS para la ejecución concurrente de tareas. El firmware implementa módulos independientes para el manejo del ventilador, lectura de sensores y comunicación web mediante un servidor HTTP que opera en formato JSON. Además, incluye servicios de red como SNTP para sincronización horaria y NVS para almacenamiento persistente de configuraciones. Cada componente se organiza en drivers y tareas especializadas que cooperan mediante colas y lógica de control, permitiendo un funcionamiento robusto y modular del sistema.

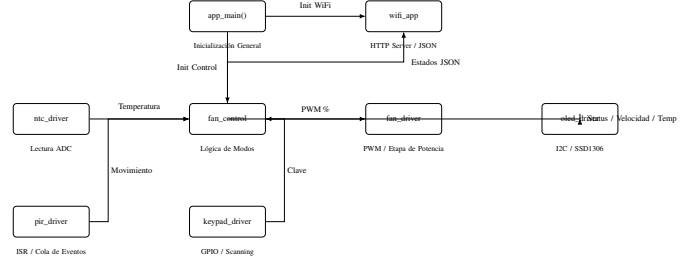


Figura 4. Arquitectura del firmware del ESP32 con integración de sensores, actuadores, pantalla OLED y teclado matricial.

IX-A. Control del ventilador

Implementado en `fan_driver.c`. Usa LEDC:

```
ledc_set_duty(...);
ledc_update_duty(...);
```

IX-B. Control de modos

- Manual: velocidad depende de slider.
- Automático: NTC define velocidad.
- Horario: triggers almacenados en NVS.

IX-C. Driver NTC

Convierte $ADC \rightarrow mV \rightarrow resistencia \rightarrow temperatura$, usando ecuación Beta.

IX-D. Driver PIR

Usa interrupciones GPIO:

```
gpio_isr_handler_add(...);
```

IX-E. API REST y NVS

El sistema ofrece endpoints para gestión de registros horarios, con almacenamiento JSON en NVS.

IX-F. WiFi y SNTP

El módulo WiFi crea AP o se conecta a red local y luego sincroniza la hora para permitir el funcionamiento del scheduler.

X. MODOS DE FUNCIONAMIENTO

X-A. Modo Manual

El usuario controla encendido y velocidad directamente.

X-B. Modo Automático por Temperatura

La velocidad aumenta con temperatura (ej. proporcional lineal 25–40°C).

X-C. Modo Automático por Horario

El usuario crea registros con:

- Hora
- Minuto
- Días de la semana

El sistema consulta cada minuto la hora actual y ejecuta acciones.

XI. COMUNICACIÓN EN TIEMPO REAL

Cada evento del usuario produce una solicitud HTTP al ESP32. El ESP32 responde de inmediato y actualiza PWM o estado, garantizando baja latencia (<10 ms localmente).

XII. RESULTADOS ESPERADOS

- Control estable y suave del ventilador mediante PWM.
- Lecturas de temperatura precisas y en tiempo real.
- Detección confiable de presencia.
- Ejecución correcta de registros horarios.
- Comunicación fluida entre interfaz y ESP32.

XIII. CONCLUSIONES

Se desarrolló un sistema embebido robusto, funcional y completo, capaz de controlar un ventilador en tiempo real mediante interfaz web. El ESP32 demostró gran capacidad para integrar sensores, comunicación, control PWM y lógica de automatización. El uso de NVS, SNTP y API REST permitió un diseño modular, escalable y confiable. Este proyecto evidencia el potencial del ESP32 en aplicaciones IoT y domótica de bajo costo.

REFERENCIAS

- [1] Espressif Systems, “ESP32 Technical Reference Manual”, 2024.
- [2] FreeRTOS Documentation.
url <https://freertos.org>
- [3] MDN Web Docs – Guía HTTP y JSON.
- [4] Beta Parameter Thermistors, Vishay Intertechnology.
- [5] HC-SR501 PIR Sensor Datasheet.